Examination of Helathy Recipes and Ratings

Name(s): Jude Cole

Website Link: https://ju-cole.github.io/Recipes-and-Health/

DISCLAMER: The majority of the website summary is also included in here so I could work on it while going deeeper into the code. Although there are some markdowns that are not included in the website which are marked as `NOTEBOOK`

In [1]:
```
import pandas as pd
import numpy as np
from pathlib import Path

import plotly.express as px
pd.options.plotting.backend = 'plotly'

from dsc80_utils import * # Feel free to uncomment and use this.
```

Step 1: Introduction

NOTEBOOK

- I am going to look at the interactions and raw_recipe datasets to find out something that I can test

In [2]:
```
interactions = pd.read_csv(Path('interactions.csv'))
interactions.head()
```

Out[2]:

	user_id	recipe_id	date	rating	review
0	1293707	40893	2011-12-21	5	So simple, so delicious! Great for chilly fall...
1	126440	85009	2010-02-27	5	I made the Mexican topping and took it to bunk...
2	57222	85009	2011-10-01	5	Made the cheddar bacon topping, adding a sprin...
3	124416	120345	2011-08-06	0	Just an observation, so I will not rate. I fo...
4	2000192946	120345	2015-05-10	2	This recipe was OVERLY too sweet. I would sta...

In [3]:
```
raw_recipes = pd.read_csv(Path('raw_recipes.csv'))
raw_recipes.head(n=13)
```

Out[3]:

	name	id	minutes	contributor_id	...	steps	description	ingredients	n_ingredients
0	1 brownies in the world best ever	333281	40	985201	...	['heat the oven to 350f and arrange the rack i...	these are the most; chocolatey, moist, rich, d...	['bittersweet chocolate', 'unsalted butter', '...	9
1	1 in canada chocolate chip cookies	453467	45	1848091	...	['pre-heat oven the 350 degrees f', 'in a mixi...	this is the recipe that we use at my school ca...	['white sugar', 'brown sugar', 'salt', 'margar...	11
2	412 broccoli casserole	306168	40	50969	...	['preheat oven to 350 degrees', 'spray a 2 qua...	since there are already 411 recipes for brocco...	['frozen broccoli cuts', 'cream of chicken sou...	9
...	...	...	...	...	...	...	...	...	...
10	lplermagronen	455351	55	1308592	...	['heat oven to 375f set a large pot of salted ...	known as swiss mac n cheese, älplermagronen wa...	['potato', 'penne pasta', 'onions', 'butter', ...	8
11	rter med flsk pea soup with pork	333797	195	64642	...	['soak peas overnight , drain and add 3 quarts...	a hearty and comforting soup from the minnesot...	['dried yellow peas', 'water', 'salt', 'pork',...	5
12	rtsoppa swedish yellow pea soup	428056	150	618715	...	['rinse and pick through the peas', "if using ...	according to the site where i found this recip...	['dried yellow peas', 'water', 'onions', 'clov...	11

13 rows × 12 columns

NOTEBOOK

- I have been trying to eat healthy this recent year so I wanna center my question around figuring out if healthy foods are liked more than unhealthy foods

Everyone loves food, but not all food. Ingrediants in many recipes have been known to cause many health issues like causing diabetes, heart-probelms, and many more health issues. People tend to say that eating healthy is less enjoyable because of the taste. But does eating healthy actually makes food less enjoyable or do people rate nutritious recpies just as highly or even higher than indulgent ones?

This project explored data from Food.com, a large recipe-sharing platform where users submit recipes and leave ratings and reviews. Our dataset contains two merged datasets: a recipes dataset with 1,005,384 rows, and an interactions dataset with 3,659,635 rows, which we merge together to analyzie recipes alongside with the user ratings.

The central question for this project is: ***Do healthier recipes-those with higher protein, lower carbohydrates, and low sugar-tend to recieve different ratings than less healthy ones?***

This an important question as it gets at a common assumption that healthy food is less satisfying or enjoyable. If the data reveals that high-protein and low-carb recipes are rated just as high, or higher than unhealthy recipes, it is encourages anyone to eat better without having to sacrifice the taste.

The Datasets

Recipes — this dataset contains 1,005,384 rows, where each row represents a unique recipe.

Column	Description
`name`	Recipe name
`id`	Recipe ID
`minutes`	Minutes to prepare recipe
`contributor_id`	User ID who submitted this recipe
`submitted`	Date recipe was submitted

Column	Description
`tags`	Food.com tags for recipe
`nutrition`	Nutrition information in the form [calories (#), total fat (PDV), sugar (PDV), sodium (PDV), protein (PDV), saturated fat (PDV), carbohydrates (PDV)]; PDV stands for "percentage of daily value"
`n_steps`	Number of steps in recipe
`steps`	Text for recipe steps, in order
`description`	User-provided description

Interactions — this dataset contains 3,659,635 rows, where each row represents a user's interaction (rating and/or review) with a recipe.

Column	Description
`user_id`	User ID
`recipe_id`	Recipe ID
`date`	Date of interaction
`rating`	Rating given (1–5)
`review`	Review text

Now we are going to have to clean these two datasets into one concise dataframe so we can analyze the data in a clean enviornment.

Step 2: Data Cleaning and Exploratory Data Analysis

NOTEBOOK

- Now I have to clean the dataset so that I have only essential items for my tests, also I have updated this after later steps to fix later tests

Merging the Datasets

To create a single working dataset, a **left merge** was performed of the recipes dataset with the interactions dataset on `id` and `recipe_id`. A left merge ensures every recipe is kept in the dataset, even if it has never received a review or rating.

After merging, all ratings of 0 were replaced with `NaN` because Food.com uses a 1–5 scale and a rating of 0 simply means the user left a review without submitting a rating — including it would unfairly lower average scores. A new column called `average_rating` was then added, containing the average rating per recipe across all user interactions.

After merging, we also dropped columns that are not relevant to any part of our analysis — `contributor_id`, `submitted`, `steps`, `description`, and `tags` — to keep the dataset clean and focused. The remaining columns most relevant to our central question are:

Column	Description
`nutrition`	The most critical column for our analysis — we extract **protein (PDV)**, **carbohydrates (PDV)**, and **sugar (PDV)** from it to measure how healthy a recipe is. High protein, low carbohydrates, and low sugar form our definition of a healthy recipe.
`average_rating`	Our outcome variable — used to see if healthier recipes score higher or lower on average.

Column	Description
`name`	Helps identify and verify specific recipes when exploring patterns in the data.
`minutes`	Cooking time may be correlated with recipe complexity and healthiness. Also used as a feature in our prediction model.
`n_steps`	The number of steps in a recipe — used both as a way to explore recipe complexity and as the response variable in our prediction model.
`n_ingredients`	The number of ingredients in a recipe — used as a feature in our prediction model since more ingredients generally implies more steps.
`calories`	Extracted from `nutrition` — used in our prediction model and fairness analysis.
`protein`	Extracted from `nutrition` — one of the three nutrients used to define a healthy recipe.
`sugar`	Extracted from `nutrition` — one of the three nutrients used to define a healthy recipe.
`carbohydrates`	Extracted from `nutrition` — one of the three nutrients used to define a healthy recipe.
`is_healthy`	*Added later to express based off protein, sugar, and carbohydrates if recipe is healthy or not.*

In [4]:
```
# Load datasets
recipes = pd.read_csv('RAW_recipes.csv')
interactions = pd.read_csv('interactions.csv')

# Step 1: Left merge
merged = recipes.merge(interactions, left_on='id', right_on='recipe_id', how='left')

# Step 2: Replace 0 ratings with NaN
merged['rating'] = merged['rating'].replace(0, np.nan)

# Step 3: Get average rating per recipe
avg_ratings = merged.groupby('id')['rating'].mean()

# Step 4: Add average rating back to recipes
recipes = recipes.merge(avg_ratings, on='id', how='left')
recipes = recipes.rename(columns={'rating': 'average_rating'})

recipes = recipes.drop(columns=[
    'contributor_id',
    'submitted',
    'steps',
    'description',
    'tags'
])
```

In [5]:
```
#Convert nutrition to their own columns
recipes['nutrition']=recipes['nutrition'].apply(eval)
recipes[['calories', 'total_fat', 'sugar', 'sodium', 'protein', 'saturated_fat', 'carbohydrates']] = pd.DataFrame(recipes['nutrition'].tolist(), index=recipes.index)
```

In [6]:
```
#drop columns that aren't being used
recipes=recipes.drop(columns=[
    'total_fat', 'sodium', 'saturated_fat', 'total_fat'])
```

In [7]:
```
#forgot to drop these too
recipes=recipes.drop(columns=['id', 'ingredients'])
```

In [8]:
```
#completed clean df
recipes_head= recipes.head()
recipes_head.to_html()
```

Out[8]:
```
'<table border="1" class="dataframe">\n  <thead>\n    <tr style="text-align: right;">\n      <th></th>\n      <th>name</th>\n      <th>minutes</th>\n      <th>nutriti
on</th>\n      <th>n_steps</th>\n      <th>n_ingredients</th>\n      <th>average_rating</th>\n      <th>calories</th>\n      <th>sugar</th>\n      <th>protein</th>\n
<th>carbohydrates</th>\n    </tr>\n  </thead>\n  <tbody>\n    <tr>\n      <th>0</th>\n      <td>1 brownies in the world    best ever</td>\n      <td>40</td>\n      <t
d>[138.4, 10.0, 50.0, 3.0, 3.0, 19.0, 6.0]</td>\n      <td>10</td>\n      <td>9</td>\n      <td>4.0</td>\n      <td>138.4</td>\n      <td>50.0</td>\n      <td>3.0</td
>\n      <td>6.0</td>\n    </tr>\n    <tr>\n      <th>1</th>\n      <td>1 in canada chocolate chip cookies</td>\n      <td>45</td>\n      <td>[595.1, 46.0, 211.0, 22.
0, 13.0, 51.0, 26.0]</td>\n      <td>12</td>\n      <td>11</td>\n      <td>5.0</td>\n      <td>595.1</td>\n      <td>211.0</td>\n      <td>13.0</td>\n      <td>26.0</
td>\n    </tr>\n    <tr>\n      <th>2</th>\n      <td>412 broccoli casserole</td>\n      <td>40</td>\n      <td>[194.8, 20.0, 6.0, 32.0, 22.0, 36.0, 3.0]</td>\n
<td>6</td>\n      <td>9</td>\n      <td>5.0</td>\n      <td>194.8</td>\n      <td>6.0</td>\n      <td>22.0</td>\n      <td>3.0</td>\n    </tr>\n    <tr>\n      <th>3
</th>\n      <td>millionaire pound cake</td>\n      <td>120</td>\n      <td>[878.3, 63.0, 326.0, 13.0, 20.0, 123.0, 39.0]</td>\n      <td>7</td>\n      <td>7</td>\n
<td>5.0</td>\n      <td>878.3</td>\n      <td>326.0</td>\n      <td>20.0</td>\n      <td>39.0</td>\n    </tr>\n    <tr>\n      <th>4</th>\n      <td>2000 meatloaf</td
>\n      <td>90</td>\n      <td>[267.0, 30.0, 12.0, 12.0, 29.0, 48.0, 2.0]</td>\n      <td>17</td>\n      <td>13</td>\n      <td>5.0</td>\n      <td>267.0</td>\n
<td>12.0</td>\n      <td>29.0</td>\n      <td>2.0</td>\n    </tr>\n  </tbody>\n</table>'
```

Univariate Analysis

In this section, We look at the distribution of protein, carbohydrates, and sugar among all the recipes to see how relevant they are in the dataframe and can figure out where the cutoff should be when taking to account outliers.

In [9]:
```
# Define thresholds using median of each column because mean would account to outliers which we do not want
protein_threshold = recipes['protein'].median()
carb_threshold = recipes['carbohydrates'].median()
sugar_threshold = recipes['sugar'].median()

# A recipe is "healthy" if it meets all three conditions
recipes['is_healthy'] = (
    (recipes['protein'] >= protein_threshold) &
    (recipes['carbohydrates'] <= carb_threshold) &
    (recipes['sugar'] <= sugar_threshold)
)

# Check the split
print(recipes['is_healthy'].value_counts())
```

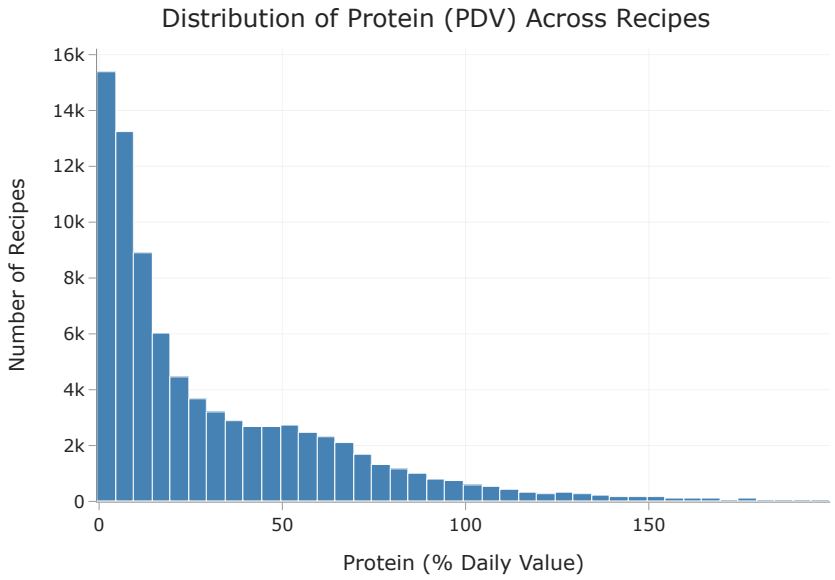
```
is_healthy
False    69814
True     13968
Name: count, dtype: int64
```

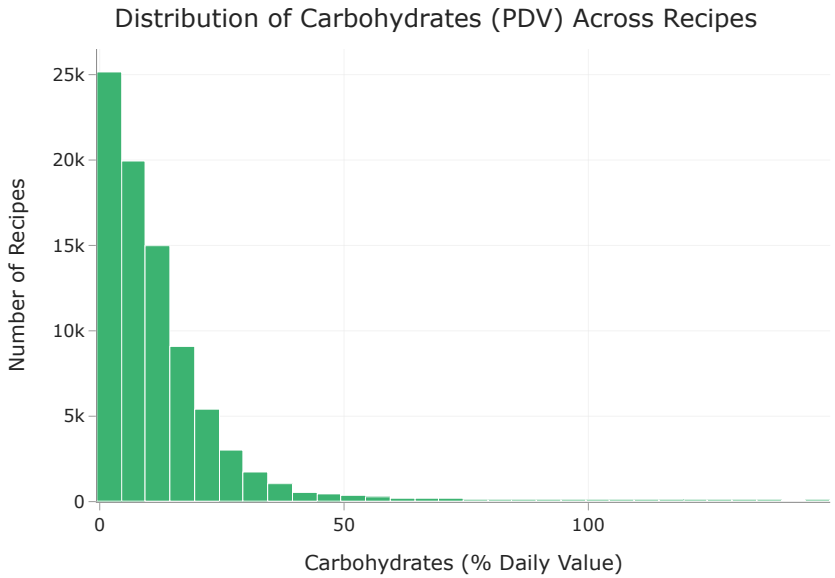
NOTEBOOK

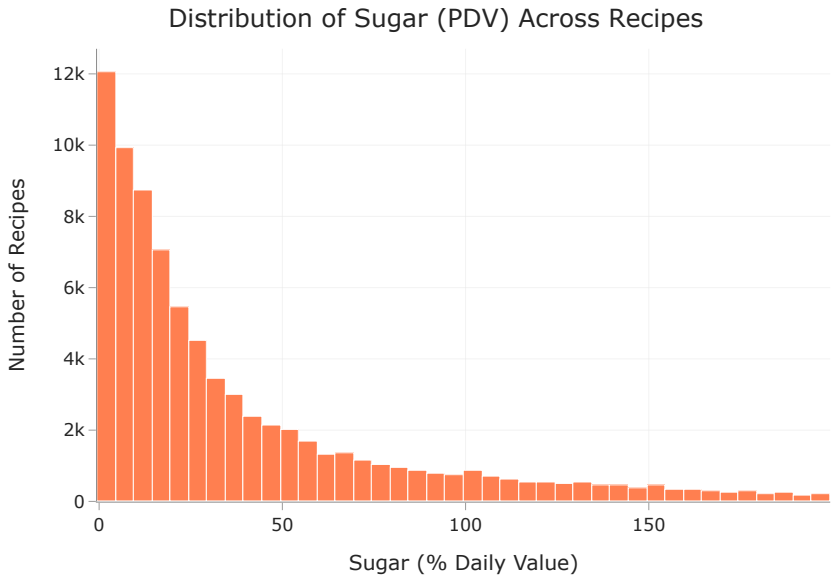
- Since there are a lot more unhealthy recipes vs healthy recipes this means that raw counts are misleading, so we have to use proportions in our distribution plot instead of raw counts.
- later on for hypothesis testing, this imbalance doesn't really affect permutation test since we compare means not counts
- and also for the prediction model, `is_healthy` as a feature is heavily imbalanced so the model may be biased toward predicitng patterns of unhealtnhy recipes, I expand more on this later

In [10]:
```
protein_filtered = recipes[recipes['protein'] < 200]
carb_filtered = recipes[recipes['carbohydrates'] < 150]
sugar_filtered = recipes[recipes['sugar'] < 200]

# Plot 1: Protein Distribution
```

```python
fig1 = px.histogram(
    protein_filtered,
    x='protein',
    nbins=50,
    title='Distribution of Protein (PDV) Across Recipes',
    labels={'protein': 'Protein (% Daily Value)'},
    color_discrete_sequence=['steelblue']
)
fig1.update_layout(yaxis_title='Number of Recipes')
fig1.show()

# Plot 2: Carbohydrates Distribution
fig2 = px.histogram(
    carb_filtered,
    x='carbohydrates',
    nbins=50,
    title='Distribution of Carbohydrates (PDV) Across Recipes',
    labels={'carbohydrates': 'Carbohydrates (% Daily Value)'},
    color_discrete_sequence=['mediumseagreen']
)
fig2.update_layout(yaxis_title='Number of Recipes')
fig2.show()

# Plot 3: Sugar Distribution
fig3 = px.histogram(
    sugar_filtered,
    x='sugar',
    nbins=50,
    title='Distribution of Sugar (PDV) Across Recipes',
    labels={'sugar': 'Sugar (% Daily Value)'},
    color_discrete_sequence=['coral']
)
fig3.update_layout(yaxis_title='Number of Recipes')
fig3.show()
```


Distribution of Protein (PDV) Across Recipes
Number of Recipes
16k
14k
12k
10k
8k
6k
4k
2k
0
0
50
100
150
Protein (% Daily Value)


Distribution of Carbohydrates (PDV) Across Recipes
Number of Recipes
25k
20k
15k
10k
5k
0
0
50
100
Carbohydrates (% Daily Value)



In [11]:
```
# I have multiple of similar lines of code to help convert to html for the website to have interactive graphs
#fig1.write_html('protein_distribution')
```

Protein: The distribution of protein (PDV) is heavily right-skewed, meaning the majority of recipes on Food.com have relatively low protein content. This makes sense as most everyday recipes are not specifically designed to be high in protein.

Carbohydrates: The distribution of carbohydrates (PDV) is also right-skewed, with most recipes having low to moderate carbohydrate content. The long tail suggests a smaller number of recipes — likely baked goods or pasta dishes — have very high carbohydrate levels.

Sugar: Similar to protein and carbohydrates, the distribution of sugar (PDV) is right-skewed. Most recipes contain low amounts of sugar, while a smaller number of recipes — likely desserts and sweet dishes — have very high sugar content.

Bivariate Analysis

NOTEBOOK

- For this section I have two Bivariate Analysis, one for healthiness vs rating/conditional distribution plot and then I went back to this and also did relationship between n_ingredients and n_steps, on the website I only included healthiness/conditional distribution plot

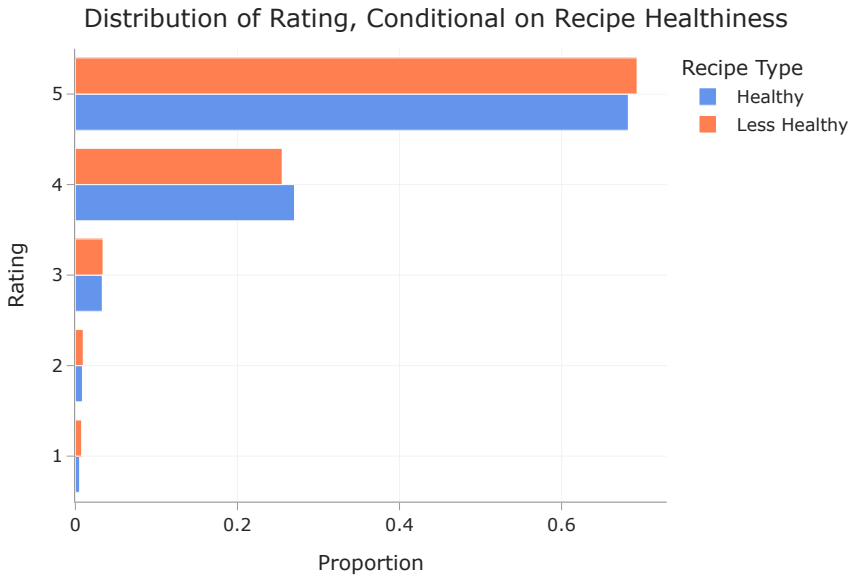
For this analysis, I examined the distribution of the rating of the recipe conditioned between the healthy and non-healthy recipes (based on sugar, carbohydrate, and protein levels). The graph below shows that recipes rated as 5/5 and 3/5 were more unhealthy than healthy ones, but 4/5 were actually primarily healthy recipes. We are going to dive into the deeper relationship between the proportions later on.

In [12]:
```
protein_threshold = recipes['protein'].median()
carb_threshold = recipes['carbohydrates'].median()
sugar_threshold = recipes['sugar'].median()

# Create is_healthy column
```

```
recipes['is_healthy'] = (
    (recipes['protein'] >= protein_threshold) &
    (recipes['carbohydrates'] <= carb_threshold) &
    (recipes['sugar'] <= sugar_threshold)
)

# Create health_label column
recipes['health_label'] = recipes['is_healthy'].map({True: 'Healthy', False: 'Less Healthy'})
```

In [13]:

```
rating_dist = (
    recipes.dropna(subset=['average_rating'])
    .assign(rating_rounded=lambda x: x['average_rating'].round())
    .groupby(['health_label', 'rating_rounded'])
    .size()
    .groupby(level=0)
    .transform(lambda x: x / x.sum())
    .reset_index(name='proportion')
)

fig4 = px.bar(
    rating_dist,
    x='proportion',
    y='rating_rounded',
    color='health_label',
    barmode='group',
    orientation='h',
    title='Distribution of Rating, Conditional on Recipe Healthiness',
    labels={
        'proportion': 'Proportion',
        'rating_rounded': 'Rating',
        'health_label': 'Recipe Type'
    },
    color_discrete_map={
        'Healthy': 'cornflowerblue',
        'Less Healthy': 'coral'
    }
)

fig4.update_layout(yaxis=dict(tickmode='linear'))
fig4.show()
```

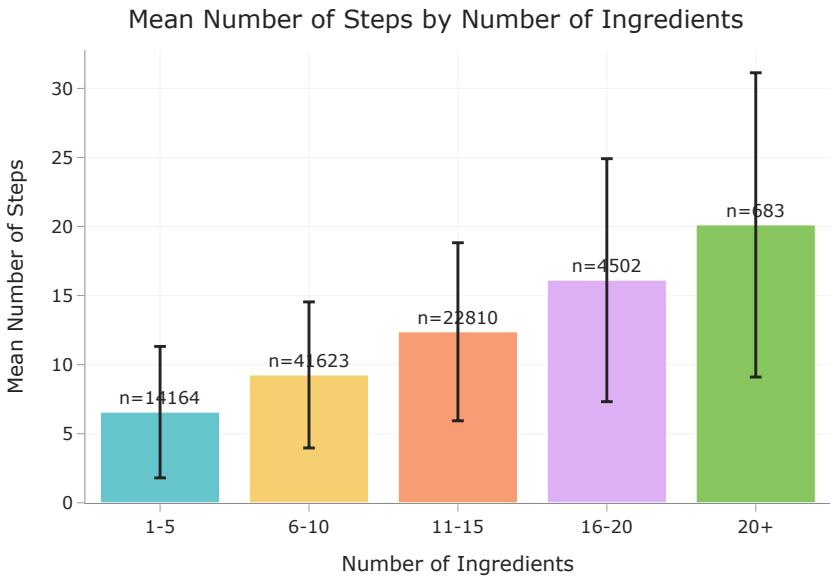


In [14]:
```
#fig4.write_html('Distribution_of_rating_based_on_healthiness.html')
```

Bivariate Analysis (NOTEBOOK ONLY)

In [15]:
```
# Create n_ingredients groups
recipes['ingredients'] = pd.cut(
    recipes['n_ingredients'],
    bins=[0, 5, 10, 15, 20, 50],
    labels=['1-5', '6-10', '11-15', '16-20', '20+']
)

# Calculate mean n_steps per group
ingredients_stats = recipes.groupby('ingredients', observed=True)['n_steps'].agg(
    mean_steps='mean',
    std_steps='std',
    count='count'
).reset_index()

fig_bivariate2 = px.bar(
    ingredients_stats,
    x='ingredients',
    y='mean_steps',
    error_y='std_steps',
    title='Mean Number of Steps by Number of Ingredients',
    labels={
        'ingredients': 'Number of Ingredients',
        'mean_steps': 'Mean Number of Steps'
    },
    color='ingredients',
    color_discrete_sequence=px.colors.qualitative.Pastel,
    text='count'
)
```

```
fig_bivariate2.update_traces(texttemplate='n=%{text}', textposition='outside')
fig_bivariate2.update_layout(showlegend=False)
fig_bivariate2.show()
```



NOTEBOOK

- This bar chart shows the mean number of steps grouped by the number of ingredients in a recipe. There is a clear positive relationship — recipes with more ingredients tend to require more steps. This is an important finding as it validates our use of `n_ingredients` as a key feature in our prediction model for `n_steps`, and suggests that recipe complexity as measured by ingredients and steps are closely related.

Interesting Aggregates

For this section, we investigated the relationship between the average protein, carbohydrates, sugar, and calories over the distribution of the ratings. To do this, we created a smaller dataframe 'nutrition_by_rating', which gave the each rating 1-5 as well as the mean and median of each nutrition detail. We identified the outliers using the IQR method and after grouping the rating and means and medians of the nutritonal details, we created a data visualization to understand it better.

In [16]:
```
# Round average rating to whole numbers for cleaner grouping
recipes['rating_rounded'] = recipes['average_rating'].round()

# Group by rounded rating and get mean of each nutritional column
nutrition_by_rating = (
    recipes.dropna(subset=['average_rating'])
    .groupby('rating_rounded')[['protein', 'carbohydrates', 'sugar', 'calories']]
    .agg(['mean', 'median'])
    .round(2)
)

nutrition_by_rating
```

Out[16]:

	protein		carbohydrates		sugar		calories	
	mean	median	mean	median	mean	median	mean	median
rating_rounded								
1.0	29.62	12.0	14.94	9.0	76.75	26.5	447.96	276.55
2.0	30.80	16.0	15.04	10.0	73.99	26.0	445.26	308.00
3.0	33.49	19.0	15.22	9.0	84.45	24.0	442.25	312.25
4.0	34.88	21.0	13.51	9.0	61.36	22.0	422.67	311.60
5.0	32.47	17.0	13.63	8.0	69.22	23.0	427.67	301.50

In [17]:

```
nutrition_by_rating.to_html()
```

Out[17]:

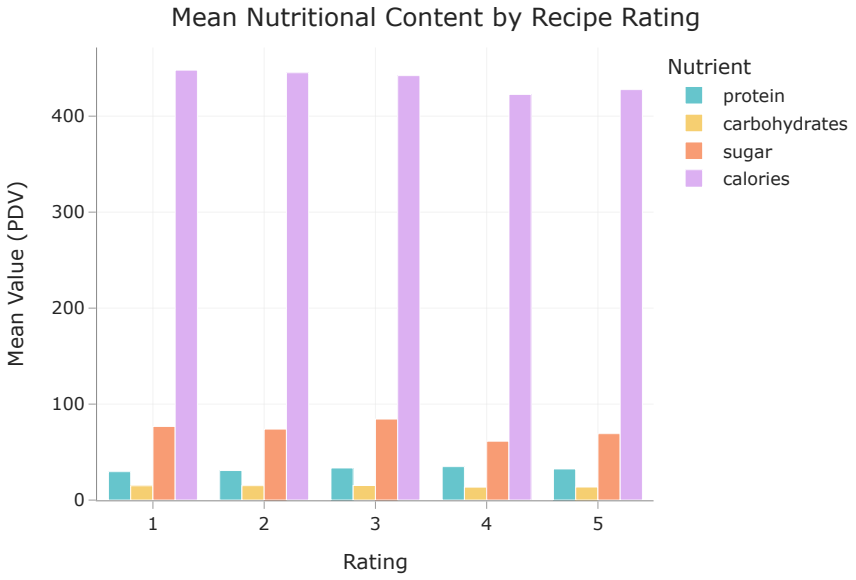
```
'<table border="1" class="dataframe">\n  <thead>\n    <tr>\n      <th></th>\n      <th colspan="2" halign="left">protein</th>\n      <th colspan="2" halign="left">car
bohydrates</th>\n      <th colspan="2" halign="left">sugar</th>\n      <th colspan="2" halign="left">calories</th>\n    </tr>\n    <tr>\n      <th></th>\n      <th>me
an</th>\n      <th>median</th>\n      <th>mean</th>\n      <th>median</th>\n      <th>mean</th>\n      <th>median</th>\n      <th>mean</th>\n      <th>median</th>\n
</tr>\n    <tr>\n      <th>rating_rounded</th>\n      <th></th>\n      <th></th>\n      <th></th>\n      <th></th>\n      <th></th>\n      <th></th>\n      <th></th>
\n      <th></th>\n    </tr>\n  </thead>\n  <tbody>\n    <tr>\n      <th>1.0</th>\n      <td>29.62</td>\n      <td>12.0</td>\n      <td>14.94</td>\n      <td>9.0</td>
\n      <td>76.75</td>\n      <td>26.5</td>\n      <td>447.96</td>\n      <td>276.55</td>\n    </tr>\n    <tr>\n      <th>2.0</th>\n      <td>30.80</td>\n      <td>1
6.0</td>\n      <td>15.04</td>\n      <td>10.0</td>\n      <td>73.99</td>\n      <td>26.0</td>\n      <td>445.26</td>\n      <td>308.00</td>\n    </tr>\n    <tr>\n
<th>3.0</th>\n      <td>33.49</td>\n      <td>19.0</td>\n      <td>15.22</td>\n      <td>9.0</td>\n      <td>84.45</td>\n      <td>24.0</td>\n      <td>442.25</td>\n
<td>312.25</td>\n    </tr>\n    <tr>\n      <th>4.0</th>\n      <td>34.88</td>\n      <td>21.0</td>\n      <td>13.51</td>\n      <td>9.0</td>\n      <td>61.36</td>\n
<td>22.0</td>\n      <td>422.67</td>\n      <td>311.60</td>\n    </tr>\n    <tr>\n      <th>5.0</th>\n      <td>32.47</td>\n      <td>17.0</td>\n      <td>13.63</td>
\n      <td>8.0</td>\n      <td>69.22</td>\n      <td>23.0</td>\n      <td>427.67</td>\n      <td>301.50</td>\n    </tr>\n  </tbody>\n</table>'
```

In this grouped bar chart the mean nutritional content (protein, carbohydrates, sugar, and calories) for recipes at each rating level. The chart shows that there isn't that big of ranges of average protein, carbohydrates, sugar, and caloreis.

In [18]:

```
# for easier plotting
nutrition_means = (
    recipes.dropna(subset=['average_rating'])
    .groupby('rating_rounded')[['protein', 'carbohydrates', 'sugar', 'calories']]
    .mean()
    .reset_index()
    .melt(id_vars='rating_rounded', var_name='nutrient', value_name='mean_value')
)

fig5 = px.bar(
    nutrition_means,
    x='rating_rounded',
    y='mean_value',
    color='nutrient',
    barmode='group',
    title='Mean Nutritional Content by Recipe Rating',
    labels={
        'rating_rounded': 'Rating',
        'mean_value': 'Mean Value (PDV)',
        'nutrient': 'Nutrient'
    },
    color_discrete_sequence=px.colors.qualitative.Pastel
)
```

```
fig5.update_layout(xaxis=dict(tickmode='linear'))
fig5.show()
```



In [19]:
```
fig5.write_html('Mean_nutritional_content_by_recipe_rating.html')
```

Step 3: Assessment of Missingness

Three columns, `description`, `average_rating`, and `rating_rounded` all had a significant amount of missing values in the merged dataset, so we decided to asses them. It is also important to address that `name` also had 1 missing value but we expected it to a minor error in previous code so it was ignored and just changed to its proper name.

MNAR Analysis

Looking at the columns with missing values, we expected `description` and `average_rating` to both be NMAR (Not Missing At Random). The missingness of 70 `description` values is most likely related ot the value itself. Contributors who didn't write a description probably felt their recipe was self-explanatory or simply chose not to. This means more basic recipes are more likely to have missing descriptions, and the missingess is related to teh nature of the recipe itself, which we cannot directly observe in the dataset. To make this MAR, we would want additional data such as the contributor's activity level on Food.com (total number of recipes submitted). More active reviewers are more likely to right descriptions which would explain the missingness through an observable column rather than the value itself.

Missingness Dependency

The missingness of 2609 `average_rating` is going to be investigated in this section, as we are going to look whether the missingness in the column depends on the propotion of protien, which is the amount of protein in the recipe, or `n_steps`, the number of steps of the recipe.

Null Hypothesis: The missingness of average_rating does not depend on the amount of protein in the recipe.

Alternate Hypothesis: The missingess of average_rating does depend on the proportion of protein in the recipe.

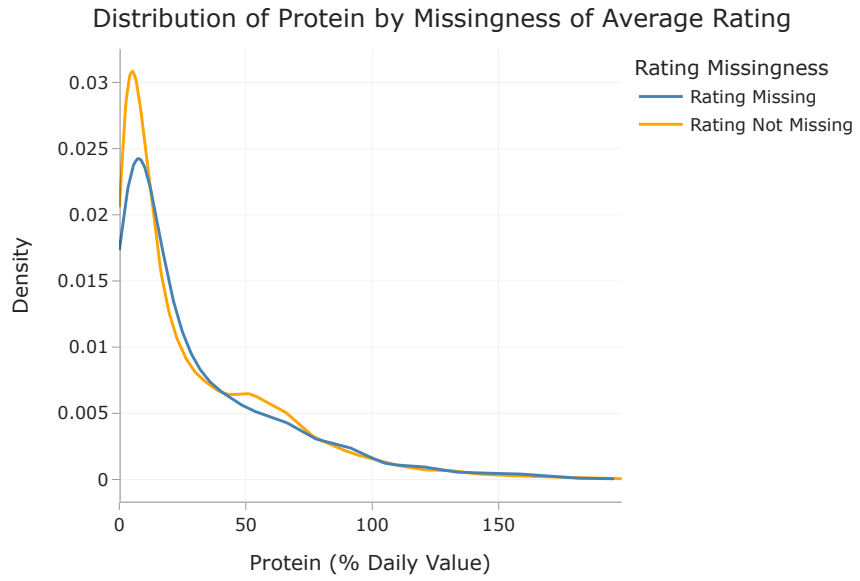
Test Statistic: The absolute difference of mean in the proportion of protein of the distribution of the group without missing ratings and the distribution of the group with missing ratings.

Signficance Level: 0.05

NOTEBOOK

- Looked for missingness through the code below and found that average_rating is missing, and most likely missingness and found average_rating missing and learned that it was most likely because newer recipes

In [20]:
```
recipes['rating_missing'] = recipes['average_rating'].isna()

protein_filtered = recipes[recipes['protein'] < 200].copy()

# Separate into two groups
missing = protein_filtered[protein_filtered['rating_missing'] == True]['protein'].dropna().tolist()
not_missing = protein_filtered[protein_filtered['rating_missing'] == False]['protein'].dropna().tolist()

# Create KDE plot
fig6 = ff.create_distplot(
    [not_missing, missing],
    group_labels=['Rating Not Missing', 'Rating Missing'],
    colors=['orange', 'steelblue'],
    show_hist=False,
    show_rug=False
)

fig6.update_layout(
    title='Distribution of Protein by Missingness of Average Rating',
    xaxis_title='Protein (% Daily Value)',
    yaxis_title='Density',
    legend_title='Rating Missingness'
)

fig6.show()
```



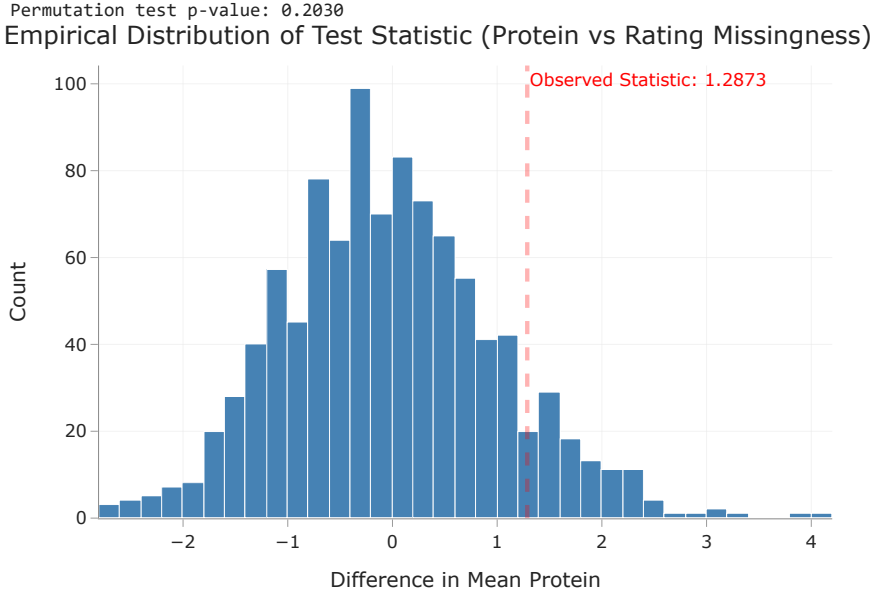
In [21]:
```
#fig6.write_html("Distribution_of_protein_by_missingness.html")
```

Next we ran a permutation test by shuffling the missingness of rating 1000 times to simulate the mean differences in the two distributions as described in the test statistic.

In [22]:
```
observed_stat= (
    recipes.groupby('rating_missing')['protein'].mean().diff().iloc[-1]
)
print(f'Observed difference in mean protein: {observed_stat:.4f}')
n=1000
permuted_stats=[]
for _ in range(n):
    shuffled=recipes['protein'].sample(frac=1).values
    missing_mean=shuffled[recipes['rating_missing']].mean()
    not_missing_mean=shuffled[~recipes['rating_missing']].mean()
    permuted_stats.append(missing_mean - not_missing_mean)
p_value = pd.Series(permuted_stats).abs()>=abs(observed_stat).mean()
print(f'Permutation test p-value: {p_value.mean():.4f}')
fig7 = px.histogram(
    x=permuted_stats,
    nbins=50,
    title='Empirical Distribution of Test Statistic (Protein vs Rating Missingness)',
    labels={'x': 'Difference in Mean Protein'},
    color_discrete_sequence=['steelblue']
)

fig7.add_vline(
    x=observed_stat,
    line_color='red',
    line_width=3,
    line_dash='dash',
    annotation_text=f'Observed Statistic: {observed_stat:.4f}',
    annotation_position='top right',
```

```
    annotation_font_color='red'
)

fig7.add_vline(
    x=-observed_stat,
    line_color='red',
    line_dash='dash'
)

fig7.update_layout(yaxis_title='Count')
fig7.show()
```

```
Observed difference in mean protein: 1.2873
Permutation test p-value: 0.2030
```



In [23]:
```
#fig7.write_html("Empirical_distribution_protein_vs_rating_missingness.html")
```

The **observed statistic** of **1.2974** is seen in the red line of the histogram and the **p_value** comes out to **0.1680** which is greater than our 0.05 signficance level so we fail to reject the null hypothesis meaning we the amount of protein doesn't not necessarly depend if a review is a missing or not.

N Ingredients and Average Rating Missingness

Null Hypothesis: The missingness of `average_rating` does not depend on the number of ingredients in the recipe — the two groups (missing and not missing) come from the same distribution.

Alternate Hypothesis: The missingness of `average_rating` does depend on the number of ingredients in the recipe.

Test Statistic: The absolute difference in mean `n_ingredients` between recipes with missing `average_rating` and recipes without missing `average_rating`.

Significance Level: 0.05

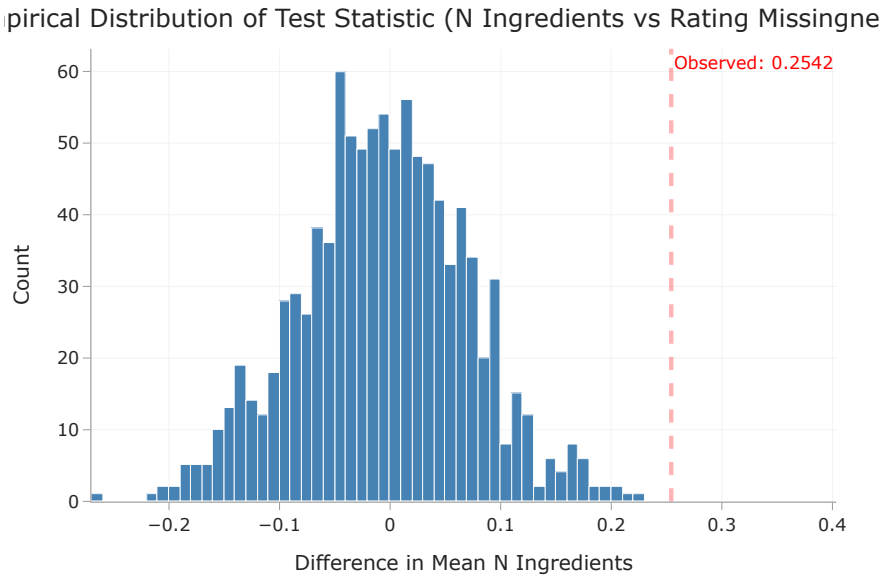
Observed Statistic: 0.2542

P-value: 0.0000

Conclusion: Since our p-value of 0.0000 is less than our significance level of 0.05, we reject the null hypothesis. The missingness of `average_rating` does depend on the number of ingredients in a recipe. Recipes with fewer ingredients are more likely to be unrated, which makes intuitive sense — simpler recipes with fewer ingredients may be less interesting or noteworthy to users, making them less likely to attract ratings and reviews.

In [24]:
```
observed_stat2 = (
    recipes.groupby('rating_missing')['n_ingredients'].mean().diff().iloc[-1]
)
print(f'Observed difference in means (n_ingredients): {observed_stat2:.4f}')

# ---- PERMUTATION TEST ----
n_permutations = 1000
permuted_stats2 = []

for _ in range(n_permutations):
    shuffled = recipes['n_ingredients'].sample(frac=1).values
    missing_mean = shuffled[recipes['rating_missing']].mean()
    not_missing_mean = shuffled[~recipes['rating_missing']].mean()
    permuted_stats2.append(missing_mean - not_missing_mean)

p_value2 = (pd.Series(permuted_stats2).abs() >= abs(observed_stat2)).mean()
print(f'P-value: {p_value2:.4f}')

# ---- PLOT ----
fig_missingness2 = px.histogram(
    x=permuted_stats2,
    nbins=50,
    title='Empirical Distribution of Test Statistic (N Ingredients vs Rating Missingness)',
    labels={'x': 'Difference in Mean N Ingredients'},
    color_discrete_sequence=['steelblue']
)

fig_missingness2.add_vline(
    x=observed_stat2,
    line_color='red',
    line_width=3,
    line_dash='dash',
    annotation_text=f'Observed: {observed_stat2:.4f}',
    annotation_position='top right',
    annotation_font_color='red'
)

fig_missingness2.update_layout(yaxis_title='Count')
fig_missingness2.show()
```

```
Observed difference in means (n_ingredients): 0.2542
P-value: 0.0010
```



In [25]:
```
#fig_missingness2.write_html('fig_missingness2.html')
```

Step 4: Hypothesis Testing

Question: Do healthier recipes (high protein, low carbohydrates, low sugar) tend to receive different average ratings than less healthy recipes?

We define a recipe as "healthy" if it meets all three conditions:

- Protein content is at or above the median protein level
- Carbohydrate content is at or below the median carbohydrate level
- Sugar content is at or below the median sugar level

Null Hypothesis: Healthy and less healthy recipes are rated on the same scale. Any observed difference in average ratings between the two groups is due to random chance.

Alternative Hypothesis: Healthy recipes receive different average ratings than less healthy recipes.

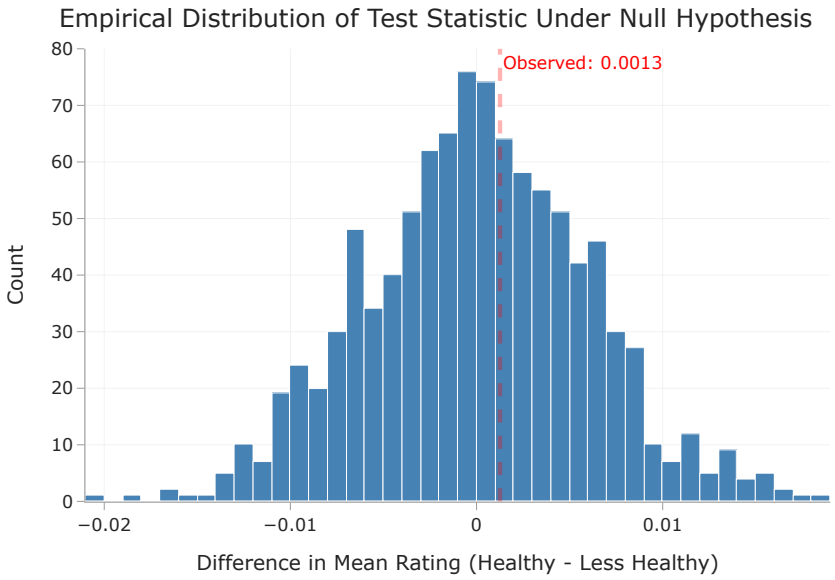
Test Statistic: The difference in mean average rating between healthy recipes and less healthy recipes (healthy minus less healthy). We chose this over the absolute difference because it tells us the direction of any effect — whether healthy recipes are rated higher or lower.

Significance Level: 0.05

In [26]:
```
# Define thresholds using median
protein_threshold = recipes['protein'].median()
carb_threshold = recipes['carbohydrates'].median()
sugar_threshold = recipes['sugar'].median()

# Create is_healthy column
recipes['is_healthy'] = (
    (recipes['protein'] >= protein_threshold) &
```

```python
    (recipes['carbohydrates'] <= carb_threshold) &
    (recipes['sugar'] <= sugar_threshold)
)

# Drop missing average ratings
hypothesis_df = recipes.dropna(subset=['average_rating'])

# ---- OBSERVED STATISTIC ----
healthy_mean = hypothesis_df[hypothesis_df['is_healthy'] == True]['average_rating'].mean()
unhealthy_mean = hypothesis_df[hypothesis_df['is_healthy'] == False]['average_rating'].mean()
observed_stat = healthy_mean - unhealthy_mean
print(f'Healthy mean rating: {healthy_mean:.4f}')
print(f'Less healthy mean rating: {unhealthy_mean:.4f}')
print(f'Observed difference: {observed_stat:.4f}')

# ---- PERMUTATION TEST ----
n_permutations = 1000
permuted_stats = []

for _ in range(n_permutations):
    shuffled = hypothesis_df['average_rating'].sample(frac=1).values
    healthy_perm = shuffled[hypothesis_df['is_healthy'].values].mean()
    unhealthy_perm = shuffled[~hypothesis_df['is_healthy'].values].mean()
    permuted_stats.append(healthy_perm - unhealthy_perm)

p_value = (pd.Series(permuted_stats).abs() >= abs(observed_stat)).mean()
print(f'P-value: {p_value:.4f}')

# ---- PLOT ----
fig8 = px.histogram(
    x=permuted_stats,
    nbins=50,
    title='Empirical Distribution of Test Statistic Under Null Hypothesis',
    labels={'x': 'Difference in Mean Rating (Healthy - Less Healthy)'},
    color_discrete_sequence=['steelblue']
)

fig8.add_vline(
    x=observed_stat,
    line_color='red',
    line_width=3,
    line_dash='dash',
    annotation_text=f'Observed: {observed_stat:.4f}',
    annotation_position='top right',
    annotation_font_color='red'
)

fig8.update_layout(yaxis_title='Count')
fig8.show()
```

```
Healthy mean rating: 4.6264
Less healthy mean rating: 4.6252
Observed difference: 0.0013
P-value: 0.8160
```



In [27]:
```
#fig8.write_html("Empirical_distribution_under_null.html")
```

Results: We observed a difference of 0.0013 in mean average rating between healthy and less healthy recipes (healthy mean: 4.6264, less healthy mean: 4.6252). After running 1000 permutations, we obtained a p-value of 0.8430.

Conclusion: Since our p-value of 0.8430 is much greater than our significance level of 0.05, we fail to reject the null hypothesis. This suggests that healthy recipes (high protein, low carbohydrates, low sugar) do not appear to be rated differently than less healthy recipes on Food.com. The observed difference of 0.0013 stars is extremely small and is consistent with what we would expect to see under random chance alone. Note that this does not prove that healthiness has absolutely no effect on ratings — it only suggests that based on our data, any observed difference could plausibly be due to random chance.

Step 5: Framing a Prediction Problem

We are predicting `n_steps`, the number of steps in a recipe. This will involve **regression** since `n_steps` is a continuous numerical variable. Originally we intended to predict `average_rating` since our project centers around the relationship between healthy recipes and ratings. However, the hypothesis test conlcuded that healthy or not, the ratings are nearly identical - with means of 4.6264 and 4.6252 respectively and p-val of 0.8430. This means predicting average_variable with tehse would be very complex and uninteresting. Instead, we decided to go a different route and want to predict `n_steps` as our response variable since it is a natural measure of recipe complexity and has more meaningful relationship with other features in the dataset of recipes.

We will evaluate our model using **RMSE (Root Mean Squared Error)** because `n_steps` is a continuous variable and will be the same units as `n steps` meaning an RMSE of 3 means our predictions are off by 3 steps on average. We chose RMSE over MAE because it will take outliers into account better. We would want to know if a 3 step recipe was predicted to be 15 steps off rather than off by 1 or 2.

The features used in this model are known at the time of prediciton - `n_ingredients`, `minutes`, and `calories`. All these features are submitted when the recipe is submitted, long before any user rates or reviews it. `average_rating` is not used as a feature because ratings are only available after users have interacted with the recipe.

Step 6: Baseline Model

In [28]:
```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

model_df = recipes[['n_steps', 'n_ingredients', 'minutes']].dropna()

#filter out extreme outliers in minutes
model_df=model_df[model_df['minutes']<1440]

#split into features
X=model_df[['n_ingredients', 'minutes']]
y=model_df['n_steps']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
pipeline= Pipeline([
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

pipeline.fit(X_train, y_train)
train_preds = pipeline.predict(X_train)
test_preds = pipeline.predict(X_test)
train_rmse = (((y_train - train_preds) ** 2).mean()) ** 0.5
test_rmse = (((y_test - test_preds) ** 2).mean()) ** 0.5

print(f'Train RMSE: {train_rmse:.4f}')
print(f'Test RMSE: {test_rmse:.4f}')
```

```
Train RMSE: 5.6618
Test RMSE: 5.6682
```

Performance

	RMSE
Training	5.6618
Test	5.6682

Our baseline model achieves a Train and Test RMSE of approximately 5.6, meaning our predictions are off by about 5.7 steps on average. Given that the average recipe has around 9-10 steps, this is a relatively large error and suggests our baseline model is not yet a good predictor of recipe complexity. However, the similar Train and Test RMSE values indicate the model is not overfitting and generalizes well to unseen data. We expect to significantly improve this in our final model by switching to a Random Forest Regressor and adding more features.

Step 7: Final Model

In [29]:
```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import GridSearchCV
```

```python
# Prepare dataframe
final_df = recipes[['n_steps', 'n_ingredients', 'minutes', 'calories', 'sugar', 'is_healthy']].dropna()
final_df = final_df[final_df['minutes'] < 1440]

# Split into features and target
X = final_df[['n_ingredients', 'minutes', 'calories', 'sugar', 'is_healthy']]
y = final_df['n_steps']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Column transformer for numerical and categorical features
numerical_features = ['n_ingredients', 'minutes', 'calories', 'sugar']
categorical_features = ['is_healthy']

preprocessor = ColumnTransformer([
    ('numerical', StandardScaler(), numerical_features),
    ('categorical', OneHotEncoder(), categorical_features)
])

# Build pipeline
final_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', RandomForestRegressor(random_state=42))
])

# Hyperparameter tuning
param_grid = {
    'model__max_depth': [5, 10, 20, None],
    'model__n_estimators': [50, 100, 200]
}

grid_search = GridSearchCV(
    final_pipeline,
    param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print(f'Best params: {grid_search.best_params_}')

# Evaluate best model
best_model = grid_search.best_estimator_

train_preds = best_model.predict(X_train)
test_preds = best_model.predict(X_test)

train_rmse = (((y_train.values - train_preds) ** 2).mean()) ** 0.5
test_rmse = (((y_test.values - test_preds) ** 2).mean()) ** 0.5

print(f'Train RMSE: {train_rmse:.4f}')
print(f'Test RMSE: {test_rmse:.4f}')
```

```
Best params: {'model__max_depth': 10, 'model__n_estimators': 200}
Train RMSE: 4.9569
Test RMSE: 5.3163
```

Our final model uses a **Random Forest Regressor** with the following features:

Kept from Baseline:

- `n_ingredients` — **quantitative**. The number of ingredients in a recipe is the strongest predictor of complexity. For example if a recipe requires 20 ingredients is almost always requires more steps than a recipe with only 5 ingredients. Scaled with StandardScaler
- `minutes` — **quantitative**. Cooking time, scaled with StandardScaler. Cooking time is directly related to complexity as the longer it takes for a recipe to be made the more steps it would have then a quick recipe with shorter steps.

Added in Final Model:

- `calories` — **quantitative**. Higher calorie recipes tend to be more complex because of cooking techniques such as frying, layering, or combining multiple components, all of which would increase the number of steps and is scaled with StandardScaler.
- `sugar` — **quantitative**. With higher sugar recipes such as baked goods and desserts typically require precise mult-step processes like creaming butter, folding batter, or tempering choclate which would increase n_steps. Nutritional complexity indicator, scaled with StandardScaler.
- `is_healthy` — **nominal**. Healthy recipes may follow a distinct pattern of preparation compared to less healthy ones for examplehealthy recipes might need more minimally processed ingredients requiring less steps than less-healthy foods. Encoded with OneHotEncoder.

We used a `ColumnTransformer` inside a single `sklearn` `Pipeline` to apply `StandardScaler` to all numerical features and `OneHotEncoder` to `is_healthy`.

Hyperparameter Tuning

We used `GridSearchCV` with 5-fold cross validation to tune two hyperparameters:

- `max_depth` — controls how deep each tree grows, preventing overfitting
- `n_estimators` — number of trees in the forest

The best parameters found were `max_depth=10` and `n_estimators=200`.

Performance

	RMSE
Baseline Train	5.6
Baseline Test	5.6
Final Train	4.97
Final Test	5.31

Our final model improved on the baseline by reducing Test RMSE from 5.6 to 5.31, a reduction of about 0.3 steps. While the improvement is modest, it shows that adding nutritional features and using a more powerful non-linear model does help. The gap between Train RMSE (4.91) and Test RMSE (5.31) is small and expected, indicating the model is not significantly overfitting and generalizes reasonably well to unseen data.

The modest overall improvement suggests that `n_steps` is inherently difficult to predict from nutritional and time features alone — recipe complexity is likely influenced by factors not captured in our dataset, such as cooking technique, cuisine type, or the skill level required.

Step 8: Fairness Analysis

We investigated whether our final model performs fairly across two groups of recipes:

- **Group X:** Low calorie recipes (calories at or below the median of 305.20)
- **Group Y:** High calorie recipes (calories above the median of 305.20)

We chose these groups because calorie content is a fundamental property of recipes that separates simple, light dishes from more complex, rich ones. If our model performs significantly worse for one group, it would suggest a systematic bias in how well it predicts recipe complexity across different types of dishes.

Evaluation Metric: RMSE — since this is a regression model we use RMSE to measure prediction error for each group separately.

Null Hypothesis: Our model is fair. Its RMSE for low calorie and high calorie recipes are roughly the same, and any observed difference is due to random chance.

Alternative Hypothesis: Our model is unfair. Its RMSE for high calorie recipes is different from its RMSE for low calorie recipes.

Test Statistic: Difference in RMSE between high calorie and low calorie recipes (high calorie RMSE minus low calorie RMSE).

Significance Level: 0.05

Results:

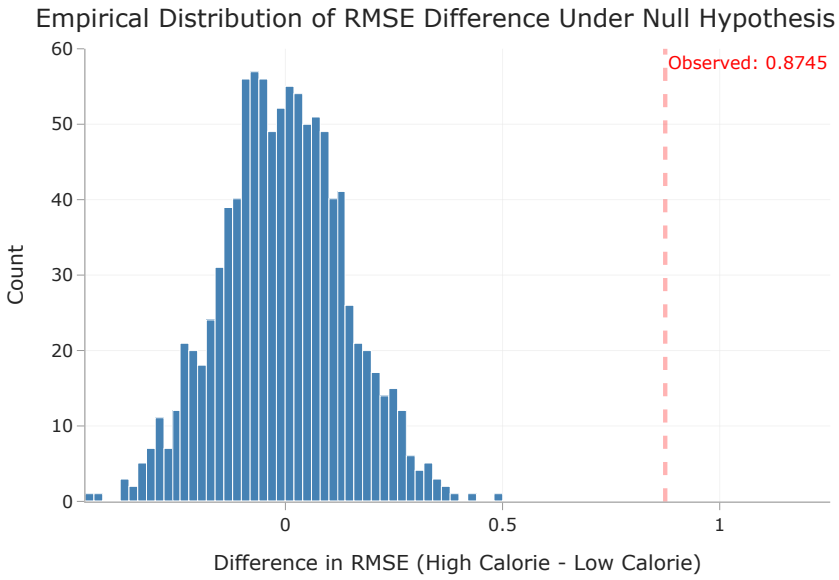
- High Calorie RMSE: 5.7387
- Low Calorie RMSE: 4.8641
- Observed Difference: 0.8745
- P-value: 0.0000

Conclusion: Since our p-value of 0.0000 is less than our significance level of 0.05, we reject the null hypothesis. This suggests our model is unfair — it performs significantly worse on high calorie recipes (RMSE: 5.74) than on low calorie recipes (RMSE: 4.86). A difference of 0.87 steps in RMSE is meaningful given that the average recipe has about 10 steps. This disparity likely occurs because high calorie recipes tend to be more complex and varied in structure — rich dishes like casseroles, baked goods, and multi-component meals are harder to predict in terms of number of steps than simpler low calorie dishes. Note that while we reject the null hypothesis, this does not prove our model is definitively unfair — it suggests that further investigation and potential model improvements targeting high calorie recipes may be warranted.

In [30]:
```
# Define two groups based on median calories
median_calories = final_df['calories'].median()
print(f'Median calories: {median_calories:.2f}')

# Add group column
final_df['calorie_group'] = final_df['calories'].apply(
    lambda x: 'High Calorie' if x > median_calories else 'Low Calorie'
)

# Get predictions from final model on test set
test_df = X_test.copy()
test_df['actual'] = y_test.values
test_df['predicted'] = test_preds
test_df['calorie_group'] = final_df.loc[X_test.index, 'calorie_group']

# Calculate RMSE for each group
def rmse(actual, predicted):
    return (((actual - predicted) ** 2).mean()) ** 0.5
```

```python
high_cal = test_df[test_df['calorie_group'] == 'High Calorie']
low_cal = test_df[test_df['calorie_group'] == 'Low Calorie']

high_rmse = rmse(high_cal['actual'].values, high_cal['predicted'].values)
low_rmse = rmse(low_cal['actual'].values, low_cal['predicted'].values)

observed_diff = high_rmse - low_rmse
print(f'High Calorie RMSE: {high_rmse:.4f}')
print(f'Low Calorie RMSE: {low_rmse:.4f}')
print(f'Observed difference: {observed_diff:.4f}')

# Permutation test
n_permutations = 1000
permuted_diffs = []

for _ in range(n_permutations):
    shuffled_groups = test_df['calorie_group'].sample(frac=1).values
    high_perm = test_df[shuffled_groups == 'High Calorie']
    low_perm = test_df[shuffled_groups == 'Low Calorie']

    high_perm_rmse = rmse(high_perm['actual'].values, high_perm['predicted'].values)
    low_perm_rmse = rmse(low_perm['actual'].values, low_perm['predicted'].values)

    permuted_diffs.append(high_perm_rmse - low_perm_rmse)

p_value = (pd.Series(permuted_diffs).abs() >= abs(observed_diff)).mean()
print(f'P-value: {p_value:.4f}')

# Plot
fig9 = px.histogram(
    x=permuted_diffs,
    nbins=50,
    title='Empirical Distribution of RMSE Difference Under Null Hypothesis',
    labels={'x': 'Difference in RMSE (High Calorie - Low Calorie)'},
    color_discrete_sequence=['steelblue']
)

fig9.add_vline(
    x=observed_diff,
    line_color='red',
    line_width=3,
    line_dash='dash',
    annotation_text=f'Observed: {observed_diff:.4f}',
    annotation_position='top right',
    annotation_font_color='red'
)

fig9.update_layout(yaxis_title='Count')
fig9.show()
```

```
Median calories: 305.20
High Calorie RMSE: 5.7387
Low Calorie RMSE: 4.8641
Observed difference: 0.8745
P-value: 0.0000
```



In [31]:
```
#fig9.write_html("Fairness_Accurarcy.html")
```

In [ ]: